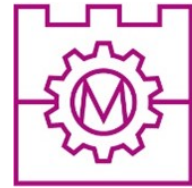




**POLITECHNIKA KRAKOWSKA**  
**im. T. Kościuszki**

Wydział Inżynierii Elektrycznej i  
Komputerowej  
**Katedra Automatyki i Informatyki E-1**



Kierunek studiów: Informatyka w inżynierii komputerowej

Specjalność: -

STUDIA NIESTACJONARNE

# **PRACA DYPLOMOWA**

INŻYNIERSKA

**Michał MISTARZ**

Mobilny system robotyczny z ramieniem wspomagany metodami uczenia  
maszynowego

Mobile robotic system with an arm supported by machine learning methods

Promotor:

dr inż. **Dariusz DOROTA**

Kraków, rok akad. 2025/2026



## OŚWIADCZENIE O SAMODZIELNYM WYKONANIU PRACY DYPLOMOWEJ

Oświadczam, że przedkładana przeze mnie praca dyplomowa **magisterska/inżynierska** została napisana przeze mnie samodzielnie. Jednocześnie oświadczam, że ww. praca:

1. nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz.U. z 2021 r. poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym, a także nie zawiera danych i informacji, które uzyskałem/am\* w sposób niedozwolony,
2. nie była wcześniej podstawą żadnej innej procedury związanej z nadawaniem tytułów zawodowych, stopni lub tytułów naukowych.

Jednocześnie wyrażam zgodę na:

1. poddanie mojej pracy kontroli za pomocą systemu Antyplagiat oraz na umieszczenie tekstu pracy w bazie danych uczelni, w celu ochrony go przed nieuprawnionym wykorzystaniem. Oświadczam, że zostałem/am\* poinformowany/a\* i wyrażam zgodę, by system Antyplagiat porównywał tekst mojej pracy z tekstem innych prac znajdujących się w bazie danych uczelni, z tekstami dostępnymi w zasobach światowego Internetu oraz z bazą porównawczą systemu Antyplagiat,
2. to, aby moja praca pozostała w bazie danych uczelni przez okres wynikający z przepisów prawa. Oświadczam, że zostałem poinformowany i wyrażam zgodę, że tekst mojej pracy stanie się elementem porównawczej bazy danych uczelni, która będzie wykorzystywana, w tym także udostępniana innym podmiotom, na zasadach określonych przez uczelnię, w celu dokonywania kontroli antyplagiatowej prac dyplomowych/doktorskich, a także innych tekstów, które powstaną w przyszłości.

.....  
podpis

3. Wyrażam zgodę na udostępnianie mojej pracy dyplomowej w Akademickim Systemie Archiwizacji Prac na PK do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych (Dz.U. z 2021 r. poz. 1062)..

TAK/NIE\*  
.....  
podpis

*Jednocześnie przyjmuję do wiadomości, że w przypadku stwierdzenia popełnienia przeze mnie czynu polegającego na przypisaniu sobie autorstwa istotnego fragmentu lub innych elementów cudzej pracy, lub ustalenia naukowego, Rektor PK stwierdzi nieważność postępowania w sprawie nadania mi tytułu zawodowego (art. 77 ust. 5 ustawy z dnia 18 lipca 2018 r. Prawo o szkolnictwie wyższym i nauce, (Dz.U. z 2021 r., poz. 478, z późn. zm.)).*

.....  
podpis



# Spis treści

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>Wykaz skrótów i oznaczeń</b>                                                   | <b>4</b>  |
| <b>1 Wstęp</b>                                                                    | <b>5</b>  |
| <b>2 Cel i zakres pracy</b>                                                       | <b>6</b>  |
| <b>3 Analiza literaturowa i przegląd technologii</b>                              | <b>8</b>  |
| 3.1 Mobilne systemy robotyczne z manipulatorem . . . . .                          | 8         |
| 3.2 Przegląd systemów napędowych i konstrukcji ramion robotycznych . . . . .      | 8         |
| 3.3 Architektury systemów sterowania robotami mobilnymi . . . . .                 | 9         |
| 3.4 Wykorzystanie wizji komputerowej w robotyce . . . . .                         | 10        |
| <b>4 Dobór i integracja komponentów sprzętowych</b>                               | <b>12</b> |
| 4.1 Założenia mechaniczne i wybór platformy jezdnej . . . . .                     | 12        |
| 4.2 Projekt i konstrukcja ramienia manipulacyjnego . . . . .                      | 13        |
| 4.3 Dobór napędów i mechanizmy wspomagające . . . . .                             | 14        |
| 4.4 Jednostka obliczeniowa, elektronika i sensoryka . . . . .                     | 16        |
| 4.5 Architektura zasilania i układ zabezpieczeń . . . . .                         | 16        |
| 4.6 Integracja mechaniczna systemu i zarządzanie okablowaniem . . . . .           | 18        |
| <b>5 Architektura systemu sterowania i komunikacja</b>                            | <b>20</b> |
| 5.1 Logiczny podział ról w architekturze rozproszonej . . . . .                   | 20        |
| 5.2 Topologia sieci i protokoły komunikacyjne . . . . .                           | 21        |
| 5.3 Architektura wieloprotokółowa i nadzorca systemu (Supervisor) . . . . .       | 22        |
| <b>6 Oprogramowanie pokładowe robota</b>                                          | <b>23</b> |
| 6.1 Sterownik serwomechanizmów i odczyt telemetry z magistrali . . . . .          | 23        |
| 6.2 Implementacja kinematyki analitycznej manipulatora . . . . .                  | 23        |
| 6.3 Ograniczenia przestrzeni roboczej i sprzętowe limity bezpieczeństwa . . . . . | 23        |
| 6.4 Akwizycja i kompresja strumienia wideo z kamery pokładowej . . . . .          | 23        |
| 6.5 Sterowanie platformą jezdnią . . . . .                                        | 23        |
| <b>7 Implementacja modułu sztucznej inteligencji</b>                              | <b>24</b> |
| 7.1 Przygotowanie i konfiguracja modelu detekcji YOLO . . . . .                   | 24        |
| 7.2 Przetwarzanie obrazu w czasie rzeczywistym i augmentacja danych . . . . .     | 24        |

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Stworzenie stacji operatorskiej</b>                                 | <b>25</b> |
| 8.1       | Projekt graficznego interfejsu użytkownika (GUI) . . . . .             | 25        |
| 8.2       | Integracja bezprzewodowego kontrolera i mapowanie przycisków . . . . . | 25        |
| 8.3       | Wizualizacja telemetrii i synchronizacja danych . . . . .              | 25        |
| <b>9</b>  | <b>Testy funkcjonalne i weryfikacja systemu</b>                        | <b>26</b> |
| 9.1       | Weryfikacja działania układu jezdnego . . . . .                        | 26        |
| 9.2       | Testy kinematyki ramienia i precyzji manipulacji . . . . .             | 26        |
| 9.3       | Ewaluacja wydajności sieciowej i opóźnień (latency) . . . . .          | 26        |
| 9.4       | Skuteczność algorytmów rozpoznawania obrazu . . . . .                  | 26        |
| <b>10</b> | <b>Wnioski i kierunki dalszego rozwoju</b>                             | <b>27</b> |
|           | <b>Summary</b>                                                         | <b>29</b> |

## Wykaz skrótów i oznaczeń

**AI** (ang. *Artificial Intelligence*) – Sztuczna inteligencja

**CNN** (ang. *Convolutional Neural Network*) – Splotowa sieć neuronowa

**CV** (ang. *Computer Vision*) – Widzenie komputerowe (maszynowe)

**DoF** (ang. *Degrees of Freedom*) – Stopnie swobody (w kontekście kinematyki ramienia)

**FPS** (ang. *Frames Per Second*) – Liczba klatek na sekundę (miara wydajności przetwarzania wideo)

**GUI** (ang. *Graphical User Interface*) – Graficzny interfejs użytkownika

**LiPo** (ang. *Lithium-Polymer*) – Typ akumulatora litowo-polimerowego

**ML** (ang. *Machine Learning*) – Uczenie maszynowe

**RPi** – Raspberry Pi (mikrokomputer pełniący funkcję jednostki obliczeniowej robota)

**RPY** (ang. *Roll, Pitch, Yaw*) – Kąty orientacji przestrzennej (przechylenie, pochylenie, odchylenie)

**UGV** (ang. *Unmanned Ground Vehicle*) – Bezzałogowy pojazd lądowy

**Venv** (ang. *Virtual Environment*) – Wirtualne środowisko (izolowane środowisko dla pakietów języka Python)

**YOLO** (ang. *You Only Look Once*) – Rodzina zaawansowanych algorytmów do detekcji obiektów w czasie rzeczywistym

# 1 Wstęp

Współczesna inżynieria oraz branża technologii informatycznych przechodzą gwałtowny rozwój w obszarze automatyzacji i robotyzacji procesów fizycznych. Tradycyjne roboty stacjonarne, aczkolwiek wysoce niezawodne w powtarzalnych zadaniach przemysłowych, napotykają istotne ograniczenia w sytuacjach wymagających elastyczności, mobilności oraz adaptacji do dynamicznie zmieniającego się otoczenia [1]. W odpowiedzi na te wyzwania, w nowoczesnym przemyśle, logistyce oraz systemach inspekcyjnych coraz większe znaczenie zyskują mobilne systemy robotyczne. Wyjątkową klasą tych urządzeń są manipulatory mobilne (ang. *mobile manipulators*), które łączą zdolność autonomicznego lub zdalnego przemieszczania się w przestrzeni z możliwością aktywnej, fizycznej interakcji z obiektami za pomocą wieloosiowych ramion robotycznych.

Kluczowym elementem warunkującym efektywność i wszechstronność takich systemów jest ich zdolność do sensorycznego postrzegania oraz inteligentnego interpretowania otaczającego środowiska. Klasyczne metody sterowania robotami, oparte wyłącznie na czujnikach odległości, krańcówkach czy sztywno zaprogramowanych trajektoriach ruchu, okazują się niewystarczające w środowiskach nieustrukturyzowanych. W miejscach, gdzie obiekty docelowe mogą zmieniać swoje położenie, orientację lub cechy wizualne, konieczne staje się wdrożenie metod sztucznej inteligencji, a w szczególności algorytmów uczenia maszynowego ukierunkowanych na wizję komputerową. Implementacja zaawansowanych modeli detekcji obiektów w czasie rzeczywistym, opartych na głębokich splotowych sieciach neuronowych z rodziny YOLO (ang. *You Only Look Once*), otwiera nowe możliwości w zakresie automatycznej identyfikacji i lokalizacji przedmiotów znajdujących się w polu widzenia kamery robota [2]. Dzięki temu operator systemu zyskuje potężne narzędzie wspomagające proces decyzyjny oraz sterowanie egzekucyjne.

Projektowanie i budowa mobilnego systemu robotycznego z ramieniem manipulacyjnym wymaga zaawansowanej synergii wiedzy z zakresu mechaniki, elektroniki, teorii sterowania oraz inżynierii oprogramowania. Istotnym wyzwaniem inżynierskim na etapie konstrukcyjnym jest nie tylko optymalny dobór komponentów wykonawczych i jednostek obliczeniowych, ale przede wszystkim opracowanie niezawodnej, rozproszonej architektury systemu sterowania. Współczesne rozwiązania bardzo często wykorzystują koncepcję podziału zadań obliczeniowych pomiędzy mikroprocesorowe układy wbudowane (lokalizowane bezpośrednio na platformie robota) a zewnętrzne stacje operatorskie dysponujące dedykowanymi akceleratorami graficznymi do przetwarzania sieci neuronowych. W takim ujęciu parametrem krytycznym staje się zapewnienie stabilnej, bezprzewodowej warstwy komunikacyjnej, która musi charakteryzować się minimalnymi opóźnieniami w transmisji danych sterujących oraz strumienia wideo.

Tematyka niniejszej pracy inżynierskiej wpisuje się bezpośrednio w aktualne trendy badawcze i aplikacyjne współczesnej informatyki stosowanej oraz robotyki. Połączenie gąsienicowej platformy jezdnej, sześćoosiowego ramienia manipulacyjnego wykonanego w technologii druku 3D oraz modułu detekcji wizyjnej opartego na sztucznej inteligencji stanowi interdyscyplinarny problem inżynierski. Praktyczna realizacja oraz analiza takiego systemu pozwalają na zbadanie ograniczeń sprzętowych, optymalizację algorytmów kinematycznych pod kątem wydajności systemów wbudowanych oraz ocenę skuteczności algorytmów uczenia maszynowego w rzeczywistych warunkach operacyjnych.



## 2 Cel i zakres pracy

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie i realizacja mobilnego systemu robotycznego wyposażonego w sześcioposiowe ramię manipulacyjne. System ten ma umożliwiać zdalne sterowanie manualne oraz zapewniać wspomaganie operatora poprzez wizyjną klasyfikację obiektów w czasie rzeczywistym z wykorzystaniem nowoczesnych metod uczenia maszynowego.

Aby zrealizować postawiony cel, zdefiniowano szczegółowy zakres prac, obejmujący następujące zadania:

- analizę literaturową i przegląd technologii dotyczących mobilnych robotów z ramieniem, systemów sterowania oraz algorytmów uczenia maszynowego (w szczególności detekcji obiektów w czasie rzeczywistym),
- dobór i integrację komponentów sprzętowych, w tym platformy jezdnej, ramienia, jednostki obliczeniowej oraz sensorów wizyjnych,
- opracowanie architektury systemu sterowania, uwzględniającej komunikację bezprzewodową pomiędzy robotem a stacją operatorską,
- implementację oprogramowania wbudowanego odpowiedzialnego za obsługę silników, kinematykę ramienia oraz przesył strumienia wideo,
- implementację modułu uczenia maszynowego do detekcji i klasyfikacji obiektów w polu widzenia kamery,
- stworzenie aplikacji klienckiej służącej do zdalnego sterowania robotem, podglądu z kamery oraz wizualizacji wyników detekcji,
- przeprowadzenie testów funkcjonalnych weryfikujących poprawność działania układu jezdnego, precyzję chwytania obiektów oraz skuteczność algorytmów rozpoznawania obrazu.

W ramach projektu określono również rygorystyczne wymagania funkcjonalne i нефункционалне. Zbudowany system musi umożliwiać intuicyjne sterowanie za pomocą bezprzewodowego kontrolera oraz dedykowanej aplikacji, realizować procedury kalibracji i bazowania osi (homing), a także chwycić i przenosić proste obiekty. Dodatkowo wymagane jest zapewnienie wysokiej responsywności interakcji, bezpieczeństwa mechanicznego i logicznego oraz szczegółowego logowania stanu pracy całego układu.

Do realizacji projektu wykorzystano szereg specjalistycznych narzędzi informatycznych oraz sprzętowych. Projekty trójwymiarowych modeli elementów mechanicznych ramienia wykonano w oprogramowaniu CAD Autodesk Fusion, a do ich fizycznego wytworzenia wykorzystano drukarkę 3D Bambu Lab P1S wraz ze środowiskiem przygotowania druku Bambu Studio. Warstwę programową aplikacji klienckiej i węzła obliczeniowego zaimplementowano w języku Python z wykorzystaniem edytora Visual Studio Code (VS Code). Oprogramowanie sprzętowe (firmware) sterownika serwomechanizmów firmy Waveshare napisano w języku C++ w środowisku Arduino IDE. W procesie tworzenia dokumentacji technicznej oraz formatowania niniejszej pracy dyplomowej posłużono się systemem składu tekstu  $\text{\LaTeX}$ .

Niniejsza praca składa się z dziesięciu głównych rozdziałów. W pierwszej części omówiono zagadnienia teoretyczne i przegląd dostępnych technologii. W kolejnych etapach przedstawiono

proces doboru komponentów sprzętowych, architekturę opracowanego systemu sterowania, a także szczegóły implementacji oprogramowania wbudowanego oraz modułu uczenia maszynowego. Ostatnie sekcje pracy poświęcono opisowi autorskiej aplikacji klienckiej oraz wynikom przeprowadzonych testów weryfikujących założenia projektowe. Podsumowanie realizacji celów zawarto we wnioskach końcowych.

### 3 Analiza literaturowa i przegląd technologii

Rozwój robotyki mobilnej oraz metod sztucznej inteligencji doprowadził do powstania zaawansowanych systemów zdolnych do pracy w dynamicznie zmieniających się środowiskach. Niniejszy rozdział stanowi wprowadzenie teoretyczne do zagadnień poruszanych w pracy. Przedstawiono w nim przegląd literatury oraz analizę aktualnych technologii związanych z mobilnymi manipulatorami, architekturami systemów sterowania, a także wykorzystaniem wizji komputerowej w układach robotycznych.

#### 3.1 Mobilne systemy robotyczne z manipulatorem

Mobilne systemy robotyczne z manipulatorem (ang. *mobile manipulators*) stanowią klasę urządzeń integrujących w sobie zalety dwóch tradycyjnie oddzielnych dziedzin robotyki: mobilnych platform jezdnych oraz ramion robotycznych [3]. Konwencjonalne manipulatory przemysłowe charakteryzują się wysoką precyzją, ładownością oraz szybkością działania, jednak ich przestrzeń robocza jest ściśle ograniczona do obszaru fizycznego zasięgu mechanizmu. Z kolei mobilne platformy jezdne oferują swobodę nawigacji na rozległych terenach, lecz standardowo pozbawione są możliwości bezpośredniej, fizycznej interakcji z otoczeniem.

Połączenie tych technologii pozwala na stworzenie systemu o teoretycznie nieograniczonej przestrzeni roboczej. Mobilny manipulator jest w stanie zdalnie bądź autonomicznie przemieścić się w docelowe miejsce operacji, a następnie wykorzystać ramię do wykonania zadania manipulacyjnego, takiego jak chwytanie, podnoszenie czy przenoszenie obiektów docelowych. Systemy tego typu znajdują szerokie zastosowanie w nowoczesnej logistyce (np. automatyczny *pick-and-place*), operacjach inspekcyjnych w środowiskach niebezpiecznych dla człowieka oraz w rolnictwie precyzyjnym.

Współczesna literatura dzieli mobilne manipulatory na kilka kategorii w zależności od zastosowanego układu lokomocji. W zastosowaniach wewnętrznych najczęściej wykorzystuje się platformy kołowe (w tym koła wielokierunkowe typu Mecanum). Z kolei w terenie o nieustrukturyzowanym podłożu, gdzie wymagane jest pokonywanie przeszkód i nierówności, znacznie lepsze właściwości trakcyjne wykazują platformy gąsienicowe.

Projektowanie i sterowanie mobilnym manipulatorem wiąże się ze znacznymi wyzwaniami inżynierskimi. Środek ciężkości (ang. *Center of Gravity* - CoG) całego systemu zmienia się dynamicznie w zależności od aktualnej konfiguracji przegubów ramienia oraz masy przenoszonego ładunku. Wymaga to implementacji solidnych rozwiązań mechanicznych, takich jak odpowiednie łożyskowanie podstawy czy mechanizmy kompensujące momenty sił, a także projektowania algorytmów sterowania uwzględniających bezpieczeństwo i stabilność przed wywróceniem całej konstrukcji.

#### 3.2 Przegląd systemów napędowych i konstrukcji ramion robotycznych

Zasadniczym elementem każdego manipulatora jest jego układ napędowy oraz struktura kinematyczna. W literaturze przedmiotu wyróżnia się kilka podstawowych typów aktuatorów stosowanych w robotyce. Najpopularniejsze z nich to silniki krokowe, bezszczotkowe silniki prądu stałego (BLDC) oraz zintegrowane serwomechanizmy [4]. Silniki krokowe zapewniają precyzyjne stero-

wanie w pętli otwartej, jednak charakteryzują się spadkiem momentu obrotowego przy wyższych prędkościach. Z kolei nowoczesne serwomechanizmy, zwłaszcza te komunikujące się za pośrednictwem cyfrowych magistrali szeregowych, oferują pracę w zamkniętej pętli sprzężenia zwrotnego. Pozwala to na precyzyjną kontrolę pozycji oraz monitorowanie parametrów telemetrycznych (np. poboru prądu, temperatury), co jest kluczowe dla bezpieczeństwa i diagnostyki systemów zrobotyzowanych [3].

Struktura mechaniczna manipulatora definiowana jest przez liczbę jego stopni swobody (ang. *Degrees of Freedom* – DoF). Aby końcówka robocza mogła osiągnąć dowolną pozycję i orientację w trójwymiarowej przestrzeni roboczej, ramię musi dysponować co najmniej sześcioma niezależnymi osiami ruchu. Sterowanie takim układem wymaga implementacji modeli matematycznych: kinematyki prostej, określającej pozycję chwytaka na podstawie kątów w przegubach, oraz kinematyki odwrotnej, wyznaczającej wymagane kąty dla zadanej pozycji docelowej. Rozwiązania kinematyki odwrotnej dzieli się na metody numeryczne (iteracyjne) oraz analityczne (wykorzystujące zamknięte wzory matematyczne), przy czym te drugie są preferowane w systemach wbudowanych z uwagi na deterministyczny czas wykonywania obliczeń [4].

W kontekście materiałoznawstwa, tradycyjne ramiona przemysłowe konstruowane są ze stopów aluminium lub stali, co zapewnia im wysoką sztywność. Jednak w prężnie rozwijającej się dziedzinie robotyki mobilnej oraz prototypowania, coraz większą rolę odgrywają technologie przyrostowe, w tym druk 3D z wykorzystaniem nowoczesnych polimerów. Pozwala to na znaczną redukcję masy własnej manipulatora, co bezpośrednio przekłada się na mniejsze zużycie energii i dłuższą pracę na zasilaniu akumulatorowym.

Niezawodność mechaniczna wymaga również zastosowania odpowiednich węzłów kinematycznych. W zależności od przenoszonych obciążeń (promieniowych, osiowych, momentów wywracających) w przegubach stosuje się różne typy łożysk: od standardowych łożysk poprzecznych, przez łożyska stożkowe, aż po łożyska wieńcowe. Dodatkowym zagadnieniem poruszonym w projektowaniu manipulatorów o dużym zasięgu jest kompensacja grawitacji. Może być ona realizowana w sposób aktywny (poprzez ciągłą pracę silników i zwiększony pobór prądu) lub pasywny (z wykorzystaniem przeciwwag, sprężyn czy siłowników gazowych), co pozwala na znaczną redukcję obciążeń statycznych układu napędowego [3].

### **3.3 Architektury systemów sterowania robotami mobilnymi**

Architektura systemu sterowania stanowi fundament niezawodnego działania każdego robota mobilnego. Wraz ze wzrostem złożoności realizowanych zadań, w nowoczesnej robotyce odchodzi się od prostych systemów scentralizowanych na rzecz architektur rozproszonych i modułarnych [3]. W ujęciu klasycznym, system taki dzieli się na warstwę niskiego poziomu (ang. *low-level control*), odpowiedzialną za bezpośrednie wysterowanie układów napędowych, stabilizację platformy i zbieranie danych z podstawowych sensorów, oraz warstwę wysokiego poziomu (ang. *high-level control*), realizującą skomplikowane zadania decyzyjne, takie jak nawigacja, analiza otoczenia czy planowanie trajektorii manipulatora.

Implementacja zaawansowanych algorytmów, w tym systemów sztucznej inteligencji, wymaga znacznych zasobów obliczeniowych. W robotyce mobilnej stosuje się dwa główne podejścia do

tego problemu: przetwarzanie brzegowe (ang. *Edge Computing*), w którym dedykowana jednostka obliczeniowa znajduje się bezpośrednio na pokładzie robota, oraz przetwarzanie zdalne, w którym ciężar obliczeniowy delegowany jest do zewnętrznej stacji operatorskiej lub klastra obliczeniowego. Przeniesienie najbardziej wymagających procesów (np. inferencji sieci neuronowych) poza platformę mobilną pozwala na znaczące obniżenie zapotrzebowania na energię elektryczną przez robota, co ma krytyczne znaczenie dla maksymalizacji czasu pracy na zasilaniu akumulatorowym [3].

Niezbędnym elementem rozproszonego systemu sterowania i przetwarzania zdalnego jest niezawodna warstwa komunikacyjna. Zdalna teleoperacja (ang. *teleoperation*) oraz ciągle przesyłanie strumienia wideo wysokiej rozdzielczości z pokładu robota wymagają zastosowania bezprzewodowych protokołów transmisji danych o odpowiedniej przepustowości, np. opartych na standardzie Wi-Fi. Kluczowym parametrem oceny takiej architektury są opóźnienia sieciowe (ang. *latency*). Ich minimalizacja jest niezbędna dla zapewnienia bezpieczeństwa ruchu robota w przestrzeni roboczej oraz dla zachowania płynności i wysokiego poziomu ergonomii pracy operatora stacji bazowej.

### **3.4 Wykorzystanie wizji komputerowej w robotyce**

Wizja komputerowa (ang. *Computer Vision*) stanowi jeden z najszybciej rozwijających się obszarów współczesnej robotyki. Integracja kamer wideo jako głównych sensorów percepcyjnych pozwala na pozyskiwanie niezwykle bogatych w informacje danych o otoczeniu robota. W przeciwieństwie do prostych czujników odległości czy skanerów laserowych, systemy wizyjne umożliwiają nie tylko wykrywanie obecności przeszkód, ale również ich rozpoznawanie, kategoryzację oraz precyzyjną lokalizację w przestrzeni trójwymiarowej.

Klasyczne metody przetwarzania obrazów, opierające się na ręcznie projektowanych filtrach i detektorach cech, wykazują dużą wrażliwość na zmienne warunki oświetleniowe, okluzje (częściowe zasłonięcie) oraz orientację przestrzenną obiektów. Z tego powodu w ostatnich latach dominującym podejściem stało się wykorzystanie algorytmów uczenia maszynowego, a w szczególności głębokich splotowych sieci neuronowych (ang. *Convolutional Neural Networks* – CNN). Sieci te potrafią samodzielnie ekstrahować hierarchiczne cechy obrazu podczas procesu uczenia na dużych zbiorach danych, co drastycznie zwiększa ich skuteczność w środowiskach nieustrukturyzowanych [5].

W kontekście wspomagania systemów sterowania manipulatorami, krytycznym zadaniem wizyjnym jest detekcja obiektów w czasie rzeczywistym. Przełomem w tej dziedzinie okazało się opracowanie architektury YOLO (ang. *You Only Look Once*) [6]. W przeciwieństwie do tradycyjnych, dwuetapowych detektorów, które najpierw generują propozycje regionów na obrazie, a następnie poddają je klasyfikacji, YOLO przetwarza całą klatkę wideo w jednym ciągłym przebiegu sieci neuronowej (ang. *single-shot detection*). Algorytm ten traktuje detekcję jako problem regresji, jednocześnie predykując współrzędne ramek otaczających (ang. *bounding boxes*) oraz prawdopodobieństwa przynależności do poszczególnych klas.

Zastosowanie jednokrokowej detekcji charakteryzuje się bardzo wysoką wydajnością obliczeniową. Jest to cecha kluczowa w zrobotyzowanych systemach operujących w czasie rzeczywistym, gdzie liczba przetwarzanych klatek na sekundę (ang. *Frames Per Second* – FPS) bezpośrednio

wpływa na jakość i bezpieczeństwo procesu sterowania. Informacja o klasie i położeniu wykrytego obiektu na obrazie z kamery może zostać zintegrowana z modelem sterowania ramieniem robota, umożliwiając realizację złożonych zadań manipulacyjnych, takich jak precyzyjne chwytnie, sortowanie oraz śledzenie celu w ruchu.

## 4 Dobór i integracja komponentów sprzętowych

Przejście od założeń teoretycznych do fizycznej realizacji mobilnego systemu robotycznego wymagało przeprowadzenia rygorystycznego procesu doboru komponentów sprzętowych. W niniejszym rozdziale zaprezentowano architekturę sprzętową (ang. *hardware architecture*) zaprojektowanego rozwiązania. Szczegółowo omówiono wybór platformy jezdnej, autorski projekt konstrukcji ramienia manipulacyjnego, zastosowane układy napędowe, a także jednostkę obliczeniową wraz z peryferiami wizyjnymi. Ważnym aspektem poruszonym w tej części pracy jest również topologia układu zasilania oraz wdrożone zabezpieczenia elektryczne, gwarantujące stabilną i bezpieczną pracę całego systemu.

### 4.1 Założenia mechaniczne i wybór platformy jezdnej

Podstawowym zadaniem platformy jezdnej w systemie manipulatora mobilnego jest zapewnienie stabilnej bazy operacyjnej dla ramienia robota, komputera pokładowego oraz systemu zasilania. Ze względu na wysoką złożoność projektową całego układu, obejmującą przede wszystkim konstrukcję i oprogramowanie sześciokościowego ramienia manipulacyjnego oraz integrację systemu wizyjnego, zdecydowano się na wykorzystanie gotowej platformy komercyjnej (ang. *Commercial Off-The-Shelf* – COTS) jako bazy lokomocyjnej.

W ramach prac wykorzystano platformę jezdnią UGV02 firmy Waveshare. Jest to konstrukcja sześciokołowa, w której cztery koła są napędzane niezależnymi silnikami prądu stałego wyposażonymi w enkodery (konfiguracja 6x4). Takie rozwiązanie zapewnia stabilną trakcję, jednak z punktu widzenia budowy kompletnego systemu mobilnego z manipulatorem, układ ten wykazuje określone ograniczenia mechaniczne.

Istotnym czynnikiem zidentyfikowanym podczas integracji okazała się ograniczona ładowność platformy bazowej. Dodanie masy własnej sześciokościowego ramienia (wydruki 3D, łożyska), ośmiu serwomechanizmów, mikrokomputera sterującego oraz niezależnych pakietów zasilających znacząco obciąża układ napędowy. Zwiększona masa całkowita powoduje większe opory na silnikach podczas manewrowania. Z tego względu docelowe środowisko operacyjne robota zostało ściśle ograniczone do płaskich, utwardzonych powierzchni (np. posadzek wewnątrz budynków), a system nie jest przeznaczony do pracy w trudnym, nieustrukturyzowanym terenie.

Z punktu widzenia inżynierii zasilania, aby zminimalizować negatywny wpływ obciążonych silników jezdnych na resztę systemu, zastosowano architekturę izolowaną. Układ napędowy platformy UGV02 zasilany jest z całkowicie niezależnego źródła energii. Wykorzystano do tego celu dedykowany pakiet składający się z trzech wysokoprądowych ogniw litowo-jonowych (Li-Ion) w formie 18650 firmy Sony, o pojemności nominalnej 3000 mAh i napięciu 3,7 V na ogniwo.

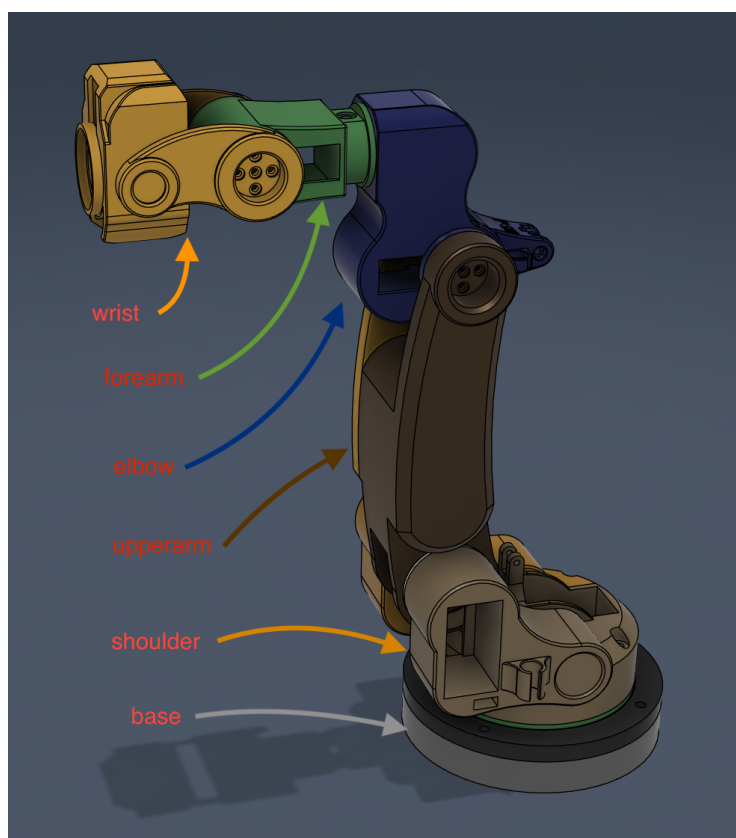
Szeregowe połączenie ogniw w module zasilania bazy zapewnia odpowiednie napięcie operacyjne dla kontrolera silników. Fizyczna separacja zasilania układu lokomocji od zasilania elektroniki i serwomechanizmów ramienia stanowi kluczową praktykę inżynierską. Silniki jezdne, szczególnie pod dużym obciążeniem wynikającym ze znacznej masy robota, generują wysokie piki prądowe, które w systemach ze wspólnym zasilaniem mogłyby prowadzić do niebezpiecznych spadków napięcia (ang. *voltage sags*) i w konsekwencji do restartów jednostki obliczeniowej lub utraty komunikacji z magistralą serwomechanizmów.

## 4.2 Projekt i konstrukcja ramienia manipulacyjnego

Zaprojektowane ramię manipulacyjne stanowi kluczowy element wykonawczy całego systemu robotycznego. W celu umożliwienia swobodnego operowania w przestrzeni trójwymiarowej oraz precyzyjnego orientowania efektora końcowego względem chwytanych obiektów, struktura kinematyczna manipulatora została zaprojektowana w oparciu o sześć stopni swobody (6-DoF).

Z uwagi na konieczność optymalizacji masy własnej przy jednoczesnym zachowaniu wysokiej sztywności konstrukcji, wszystkie elementy nośne ramienia zostały zaprojektowane w środowisku CAD, a następnie wytworzone w technologii druku 3D (FDM). Jako materiał konstrukcyjny wybrano filament ASA (Akrylonitryl-styren-akrylan), który charakteryzuje się znakomitą wytrzymałością mechaniczną oraz doskonałą adhezją międzywarstwową, co jest parametrem krytycznym w elementach przenoszących zmienne momenty zginające.

Zgodnie z przyjętą w projekcie nomenklaturą, struktura kinematyczna manipulatora została podzielona na sześć głównych członów: podstawę (ang. *base*), bark (*shoulder*), ramię właściwe (*upperarm*), łokieć (*elbow*), przedramię (*forearm*) oraz nadgarstek (*wrist*). Model CAD kompletnego ramienia, wraz z oznaczeniem poszczególnych modułów, przedstawiono na rysunku 4.1.



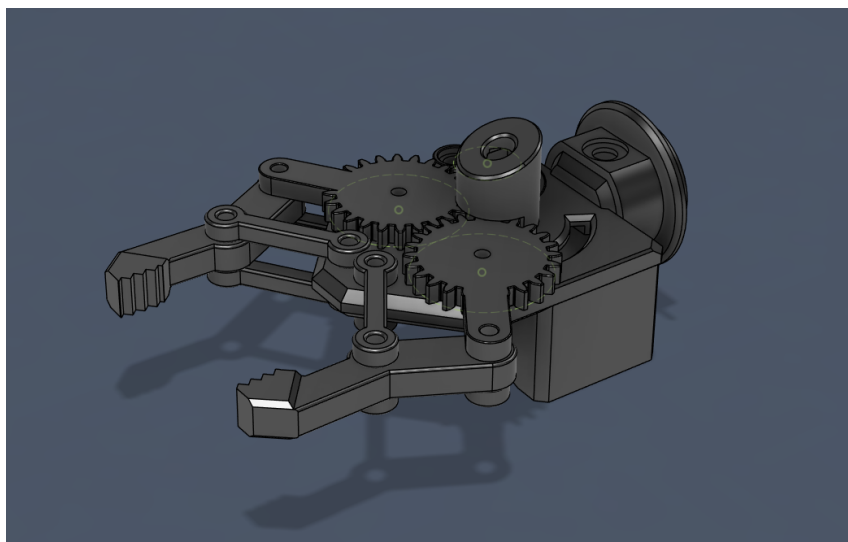
Rys. 4.1. Model CAD sześćoosiowego ramienia manipulacyjnego z podziałem na główne człony kinematyczne

Aby zapewnić płynność ruchu, zminimalizować luzy oraz bezpiecznie przenieść obciążenia statyczne i dynamiczne na strukturę nośną, wdrożono zróżnicowany system łożyskowania. W newralgicznym węźle łączącym podstawę z barkiem, narażonym na największe siły oraz momenty wywracające wynikające z maksymalnego wysięgu ramienia, zastosowano rozwiązanie hybrydowe.



Składa się ono z szerokiego łożyska wieńcowego (typu *lazy susan*) przenoszącego główne obciążenia osiowe oraz umieszczonego centralnie wewnątrz mechanizmu łożyska stożkowego. Pozostałe węzły kinematyczne ramienia zostały wyposażone w standardowe łożyska poprzeczne, co w zupełności wystarcza do poprawnego przenoszenia mniejszych obciążeń w wyższych partiach manipulatora.

Ostatnim ogniwem łańcucha kinematycznego jest efektor końcowy w postaci chwytaka dwuszczękowego. Model mechanizmu chwytającego zaprezentowano na rysunku 4.2.



Rys. 4.2. Model CAD efektora końcowego wykorzystującego przekładnię zębatą do synchronizacji szczęk

Chwytnak napędzany jest pojedynczym serwomechanizmem. Aby zapewnić równoległe i symetryczne zamykanie obu szczęk względem osi centralnej, w mechanizmie zintegrowano przekładnię zębatą. Zębátky sprzęgają ruch obu ramion chwytaka, co gwarantuje precyzyjne i powtarzalne centrowanie chwytnych obiektów, ułatwiając tym samym proces zautomatyzowanego planowania trajektorii.

### 4.3 Dobór napędów i mechanizmy wspomagające

Funkcję jednostek napędowych w zaprojektowanym manipulatorze pełnią inteligentne serwomechanizmy magistralowe ST3215 firmy Waveshare (rys. 4.3). Wybór tego konkretnego modelu podyktowany był przede wszystkim koniecznością zapewnienia bardzo wysokiego momentu obrotowego (wymaganego do poruszania wydrukowanymi członami ramienia) oraz możliwością pracy w zamkniętej pętli sprzężenia zwrotnego. Zastosowanie tak silnych napędów wiązało się jednak z koniecznym kompromisem projektowym. Ich stosunkowo duże gabaryty i masa własna wpłynęły na ostateczne rozmiary niektórych elementów układu, co jest szczególnie zauważalne w dość masywnej konstrukcji efektora końcowego (chwytaka). W całym systemie wykorzystano łącznie osiem takich aktuatorów.

Zastosowanie serwomechanizmów z interfejsem szeregowym pozwoliło na ich częściowe łączenie kaskadowe (ang. *daisy-chaining*). Chociaż specyfika kinematyczna ramienia nadal wymagała doprowadzenia głównych linii zasilających i sygnałowych do kluczowych węzłów obrotowych,



*Rys. 4.3. Serwomechanizm magistralowy ST3215 wykorzystany jako główny element napędowy ramienia*

to możliwość szeregowego spięcia blisko sąsiadujących napędów pomogła ograniczyć całkowitą ilość i złożoność okablowania wewnątrz robota. Co najważniejsze, wbudowane w serwa mikro-kontrolery umożliwiają ciągły odczyt parametrów telemetrycznych. System nadrzędny w czasie rzeczywistym monitoruje aktualną pozycję wału, napięcie zasilania, pobór prądu oraz temperaturę każdego z napędów, co stanowi podstawę dla wdrożenia mechanizmów bezpieczeństwa.

Z punktu widzenia kinematyki, ramię posiada sześć stopni swobody, jednak do realizacji ruchu wykorzystuje siedem silników (ósmy serwomechanizm odpowiada za napęd szczęk chwytaka). Z uwagi na fakt, że staw barkowy (*shoulder*) jest elementem przenoszącym największe obciążenia statyczne i dynamiczne – wynikające z masy całego ramienia oraz ładunku – zastosowano w nim konfigurację podwójną. Dwa serwomechanizmy (lewy i prawy) zostały zamontowane przeciwsobnie na jednej osi obrotu i pracują w pełni synchronicznie. Rozwiązanie to pozwala na podwojenie sumarycznego momentu obrotowego dostępnego w tym krytycznym węźle.

Mimo zastosowania wydajnych napędów oraz podwójnego serwomechanizmu w barku, praca ramienia na maksymalnym wysięgu wiąże się ze znacznym wzrostem prądu trzymania (ang. *holding current*), co mogłoby prowadzić do szybkiego przegrzewania się silników oraz drenażu akumulatora. Aby zniwelować to zjawisko, w konstrukcji zaimplementowano pasywny system kompensacji grawitacji.

Mechanizm wspomagający składa się z wytrzymałej sprężyny naciągowej, zamocowanej pomiędzy obrotową podstawą a członem łokciowym manipulatora. Kluczowym elementem tego układu jest zastosowanie śruby rzymskiej, która pełni rolę płynnego regulatora napięcia wstępnej sprężyny. Odpowiednio skalibrowana sprężyna generuje moment siły przeciwny do momentu wywołanego przez grawitację, działając jak fizyczna przeciwwaga. Dzięki temu systematycznie odciąża serwomechanizmy, pozwalając im na pracę z optymalną wydajnością prądową nawet w skrajnie rozciągniętych pozycjach manipulatora.

## 4.4 Jednostka obliczeniowa, elektronika i sensoryka

Realizacja założeń projektowych, w szczególności jednoczesna obsługa komunikacji bezprzewodowej, obliczanie kinematyki ramienia w czasie rzeczywistym oraz akwizycja strumienia wideo, wymagała zastosowania wydajnego komputera pokładowego. Jako główną jednostkę obliczeniową robota (ang. *Controller Node*) wybrano mikrokomputer Raspberry Pi 5 wyposażony w 8 GB pamięci RAM. Architektura oparta na wielordzeniowym procesorze ARM zapewnia wystarczającą moc obliczeniową do płynnego zarządzania środowiskiem uruchomieniowym bazującym na języku Python oraz do utrzymania stabilnego opóźnienia sieciowego podczas strumieniowania danych do stacji operatorskiej.

Z uwagi na specyfikę cyfrowej magistrali szeregowej wykorzystywanej przez serwomechanizmy ST3215, bezpośrednie sterowanie nimi z poziomu styków GPIO mikrokomputera Raspberry Pi byłoby nieefektywne i podatne na błędy czasowe (tzw. *jitter*). Z tego względu w architekturze elektroniki zastosowano dedykowany sterownik pośredniczący (ang. *Servo Driver*) firmy Wave-share, którego centralnym elementem jest mikrokontroler z rodziny ESP32. Układ ten pełni funkcję sprzętowego interfejsu (mostka) pomiędzy wysokopoziomowymi poleceniami wysyłanymi z Raspberry Pi a niskopoziomowym protokołem komunikacyjnym magistrali serwomechanizmów. Rozwiązanie to odciąża główną jednostkę obliczeniową i gwarantuje wysoką niezawodność odczytu telemetrii.

Kluczowym elementem sensorycznym systemu, warunkującym poprawne działanie modułu sztucznej inteligencji, jest kamera pokładowa. Do akwizycji obrazu wykorzystano kamerę internetową A4Tech PK-910H, podłączoną do komputera Raspberry Pi za pomocą interfejsu USB. Wybór tego sensora podyktowany był koniecznością uzyskania zadowalającej ostrości obrazu (rozdzielczość 1080p) oraz odpowiedniego kąta widzenia przy zachowaniu pełnej kompatybilności ze środowiskiem systemu Linux (standard UVC – *USB Video Class*).

Kamera została zamocowana na przedostatnim członie ramienia (przedramieniu), co pozwala na dynamiczną zmianę punktu widzenia w zależności od aktualnej pozycji manipulatora. Obraz pozyskiwany z sensora nie jest przetwarzany bezpośrednio na pokładzie robota – ze względu na wysokie zapotrzebowanie algorytmów wizyjnych na zasoby sprzętowe, surowy strumień wideo przesyłany jest siecią bezprzewodową do stacji operatorskiej (ang. *Compute Node*), gdzie poddawany jest docelowej analizie przez model detekcji obiektów.

## 4.5 Architektura zasilania i układ zabezpieczeń

Zapewnienie stabilnego i bezpiecznego zasilania dla układów o dużym, impulsowym poborze prądu stanowiło jedno z głównych wyzwań integracyjnych. Zgodnie z przyjętą architekturą izolowaną, układ ramienia oraz jednostka obliczeniowa zasilane są z niezależnego akumulatora litowo-polimerowego (LiPo) w konfiguracji 3S (trzy ogniwa połączone szeregowo). Pakiet ten dostarcza napięcie nominalne na poziomie 11,1 V, co stanowi wartość optymalną do zasilania wysokonapięciowych serwomechanizmów ST3215.

W celu zagwarantowania bezpieczeństwa, główna linia zasilająca (+VCC) została natychmiast za akumulatorem zabezpieczona szybkim bezpiecznikiem samochodowym o wartości 15 A. Prąd o takim natężeniu pozwala na bezproblemową, jednoczesną pracę wszystkich ośmiu serwomecha-

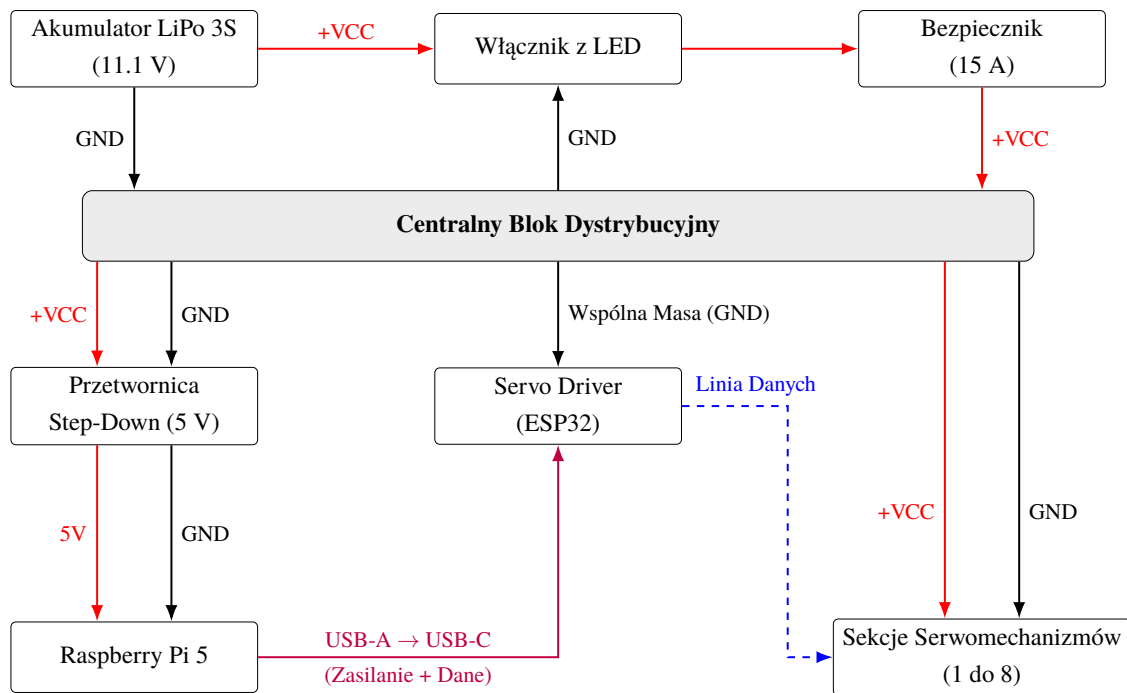
nizmów pod obciążeniem oraz mikrokomputera. Bezpośrednio za bezpiecznikiem umieszczono główny włącznik kołyskowy. Z uwagi na zintegrowane w nim podświetlenie LED, do włącznika doprowadzono również główny przewód masy (GND).

Aby uniknąć zjawiska skumulowanego spadku napięcia na cienkich przewodach, wdrożono scentralizowany system dystrybucji zasilania. Opiera się on na szybkozłączkach instalacyjnych typu Wago, które umieszczono w dolnej części bazy, tuż za włącznikiem. Pozwoliło to na podział głównej szyny prądowej na dedykowane sekcje zasilające:

- **Sekcja 1:** zasilą serwomechanizm nr 1 (obrot bazy),
- **Sekcja 2:** zasilają serwomechanizmy nr 2 i 3 (podwójny napęd stawu barkowego),
- **Sekcja 3:** zasilają serwomechanizmy nr 4 i 5 (ramię i łokieć),
- **Sekcja 4:** zasilają serwomechanizmy nr 6, 7 i 8 (przedramię, nadgarstek i chwytak).

Wewnątrz wyżej wymienionych grup zasilanie prowadzone jest kaskadowo, co stanowi optymalny kompromis pomiędzy ilością okablowania a stabilnością napięcia. Warto zaznaczyć, że sterownik serwomechanizmów (Servo Driver) nie pobiera energii z głównej szyny zasilającej LiPo – jest on zasilany niezależnie poprzez złącze USB. Aby jednak zapewnić prawidłową propagację sygnałów logicznych, przewód masy (GND) sterownika został połączony z głównym blokiem dystrybucyjnym Wago, tworząc wspólną masę dla całej instalacji. Linia sygnałowa magistrali (DATA) wychodząca z kontrolera ESP32 nie jest prowadzona ściśle kaskadowo przez wszystkie napędy. Z uwagi na ułatwienie poprowadzenia przewodów wewnątrz ramienia, zastosowano architekturę mieszaną z rozgałęzieniami, w której główna szyna danych dzieli się na odnogi doprowadzone do poszczególnych sekcji.

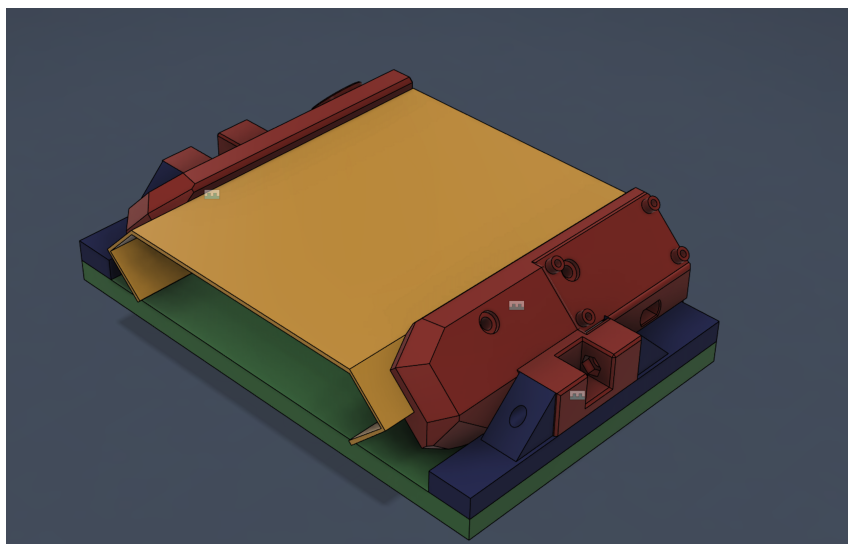
Ostatnia linia zasilająca wyprowadzona z kostki Wago dedykowana jest dla jednostki obliczeniowej. Napięcie z akumulatora LiPo przekazywane jest na moduł przetwornicy obniżającej (ang. *step-down buck converter*), która stabilizuje napięcie do poziomu 5 V, doprowadzanego następnie złączem USB-C do mikrokomputera Raspberry Pi 5. Uproszczony schemat blokowy opisanej topologii zasilania i komunikacji przedstawiono na rysunku 4.4.



Rys. 4.4. Schemat blokowy dystrybucji zasilania i komunikacji magistrali szeregowej

## 4.6 Integracja mechaniczna systemu i zarządzanie okablowaniem

Ostatnim etapem budowy warstwy sprzętowej była fizyczna integracja wszystkich opisanych wcześniej podsystemów w jedną, spójną jednostkę mobilną. W celu optymalizacji dostępnej przestrzeni oraz ułatwienia obsługi serwisowej robota, wprowadzono autorskie modyfikacje mechaniczne platformy bazowej. Na rysunku 4.5 zaprezentowano model CAD zintegrowanego modułu rozszerzającego. Kolorem zielonym oznaczono górną płaszczyznę platformy UGV02, do której przytwierdzone są fabryczne szyny montażowe (kolor niebieski). W szyny te wsunięto specjalnie zaprojektowane i wydrukowane w technologii 3D adaptery nośne (kolor czerwony). Stanowią one solidne punkty mocowania dla dodatkowego, aluminiowego pokładu (kolor żółty), dostarczanego jako opcjonalne wyposażenie podwozia.



*Rys. 4.5. Model CAD modułu rozszerzającego: zielony – pokrywa bazy, niebieski – szyny, czerwony – adaptery 3D, żółty – pokład aluminiowy*

Kluczową zaletą zaprojektowanego mechanizmu jest możliwość płynnego przesuwania całego zmontowanego modułu wzdłuż szyn, równoległe do osi podłużnej robota. Takie rozwiązanie konstrukcyjne gwarantuje bezproblemowy i szybki dostęp do włącznika zasilania platformy UGV02, fabrycznego gniazda ładowania bazy jezdnej, a także do złączy zasilających głównego akumulatora LiPo zasilającego ramię. Pozwala to na wygodne podłączenie ładowarki bez konieczności kłopotliwego demontażu ramienia manipulacyjnego lub innych elementów konstrukcyjnych.

Dodatkowy górny pokład posłużył do ergonomicznego rozmieszczenia elektroniki, w tym w szczególności sterownika serwomechanizmów (Servo Driver) firmy Waveshare opartego na mikrokontrolerze ESP32. W celu zapewnienia wysokiej niezawodności i ochrony okablowania przed przetarciami w trakcie ruchu ramienia, wszystkie główne wiązki przewodów zostały poprowadzone w elastycznych opłotach ochronnych. Zastosowanie opłotów ułatwiło segregację linii sygnałowych i zasilających, znacząco podnosząc bezpieczeństwo przeciwzwarceniowe oraz ogólną estetykę i niezawodność układu.

## 5 Architektura systemu sterowania i komunikacja

Współczesne systemy robotyczne, szczególnie te integrujące algorytmy sztucznej inteligencji z fizycznymi układami wykonawczymi, wymagają starannie zaprojektowanej architektury oprogramowania. Głównym wyzwaniem w takich projektach jest zapewnienie płynnej i szybkiej wymiany danych pomiędzy warstwą percepcyjną, modułem sterującym oraz sprzętowymi kontrolerami napędów. Niniejszy rozdział opisuje strukturę logiczną stworzonego systemu, podział ról pomiędzy jednostkami obliczeniowymi oraz zastosowane technologie komunikacyjne.

### 5.1 Logiczny podział ról w architekturze rozproszonej

Zastosowanie zaawansowanych modeli głębokich sieci neuronowych (takich jak algorytm YOLO) do analizy obrazu w czasie rzeczywistym wymaga znacznych zasobów obliczeniowych, w szczególności dostępu do wydajnych akceleratorów. Z tego względu w projekcie zrezygnowano ze scentralizowanego przetwarzania wszystkich danych wyłącznie na pokładzie robota. Zamiast tego wdrożono architekturę systemu rozproszonego. Jak zauważa się w literaturze przedmiotu, przeniesienie złożonych obliczeniowo zadań z mobilnych jednostek o ograniczonym budżecie energetycznym na zewnętrzne stacje robocze pozwala na sprawniejszą realizację algorytmów percepcji, znacząco zwiększając możliwości poznawcze układów zrobotyzowanych [7].

W zaprojektowanym systemie wydzielono dwa główne węzły (ang. *nodes*), z których każdy pełni ściśle określoną funkcję:

- **Węzeł obliczeniowy (ang. *Compute Node*) – Stacja operatorska:** Stanowi centrum percepcyjne i nadrzędny interfejs sterujący całego systemu. Rolę tę pełni zewnętrzny komputer (laptop) wyposażony w środowisko zdolne do szybkiej inferencji modelu detekcji obiektów. Jednostka ta odbiera surowy strumień wideo z robota i w czasie rzeczywistym poddaje go analizie wizyjnej, lokalizując docelowe obiekty w kadrze poprzez wyznaczanie i wizualizację ramek ograniczających (ang. *bounding boxes*). Równolegle, stacja operatorska służy do generowania i wysyłania wysokopoziomowych danych sterujących (np. współrzędnych docelowych) do robota.
- **Węzeł sterujący (ang. *Controller Node*) – Jednostka pokładowa:** Zadanie to realizowane jest przez mikrokomputer Raspberry Pi 5 znajdujący się fizycznie na platformie jezdnej. Do jego głównych zadań należy nieprzerwana akwizycja obrazu z kamery USB i strumieniowanie go przez sieć Wi-Fi do stacji operatorskiej. Równolegle jednostka ta odbiera wysokopoziomowe komendy ruchu. Wykorzystując zaimplementowany algorytm analitycznej kinematyki odwrotnej (ang. *Inverse Kinematics* – IK), Raspberry Pi przelicza otrzymane dane na docelowe kąty obrotu dla poszczególnych stawów ramienia. Z uwagi na analityczny (równaniowy) charakter modelu, obliczenia te są wysoce zoptymalizowane i realizowane w ułamku sekundy, nie obciążając znacząco procesora. Następnie wyliczone pozycje przekazywane są interfejsem szeregowym do sterownika ESP32 (Servo Driver), który realizuje ruch napędów.

Taki ścisły podział kompetencji niesie za sobą wymierne korzyści inżynierskie. Odciąża procesor jednostki pokładowej z zadań wizyjnych, pozwalając mu skupić się na obliczeniach kinema-

tycznych i utrzymaniu stabilnej komunikacji. Jednocześnie zapewnia elastyczność – stację operatorską można w dowolnym momencie zmodernizować w celu uzyskania szybszej inferencji AI, bez konieczności ingerencji w sprzęt i oprogramowanie samego robota.

## 5.2 Topologia sieci i protokoły komunikacyjne

Zapewnienie stabilnej, asynchronicznej i niskopoziomowej wymiany danych między stacją operatorską a robotem jest zadaniem krytycznym z punktu widzenia płynności sterowania. Zamiast tradycyjnego modelu opartego na surowych gniazdach TCP/UDP, w projekcie wdrożono wydajną bibliotekę asynchronicznego przesyłania wiadomości *ZeroMQ* (ZMQ). Rozwiązanie to warunkuje wysoką przepustowość, automatyczne zarządzanie retransmisją oraz oferuje wbudowane wzorce komunikacyjne, co stanowi standard w nowoczesnych, rozproszonych systemach robotycznych.

Fizyczną warstwę transportową stanowi standardowa sieć bezprzewodowa Wi-Fi (IEEE 802.11). Mikrokomputer Raspberry Pi pobiera swój lokalny adres IP i wyświetla go na module OLED, co umożliwia stacji bazowej nawiązanie bezpośredniego połączenia. Z punktu widzenia oprogramowania, zewnętrzna topologia sieciowa została podzielona na trzy niezależne, równoległe kanały komunikacyjne (strumienie), z których każdy operuje na dedykowanym porcie i wykorzystuje zoptymalizowany wzorzec przesyłu danych:

1. **Strumień Sterowania (Port 5555):** Odpowiada za przesyłanie poleceń ruchowych z węzła obliczeniowego do jednostki pokładowej. Zrealizowany jest w oparciu o wzorzec Żądanie-Odpowiedź (ang. *Request-Reply*). Stacja bazowa (klient) cyklicznie wysyła pakiety w formacie JSON, zawierające m.in. aktualny stan kontrolera (gamepada) oraz wyliczone współrzędne docelowe. Główna usługa na robocie (ang. *Arm Service*) odbiera pakiet, aplikuje go do równań kinematyki, a następnie odsyła potwierdzenie, zamykając cykl.
2. **Strumień Wideo (Port 5556):** Dedykowany wyłącznie do transmisji obrazu z pokładowej kamery USB. Działa w oparciu o wzorzec Publikuj-Subskrybuj (ang. *Publish-Subscribe*). Skrypt na Raspberry Pi przechwytuje klatki, dokonuje ich natychmiastowej kompresji do formatu JPEG za pomocą biblioteki OpenCV, a następnie publikuje surowe ciągi bajtów. Aby zminimalizować zjawisko opóźnienia wizyjnego (ang. *latency*), po stronie subskrybenta (laptopa) aktywowano sprzętową opcję ZMQ\_CONFLATE. Powoduje ona ignorowanie starszych, zalegających w buforze klatek wideo na rzecz przetwarzania wyłącznie tej najnowszej.
3. **Strumień Telemetrii (Port 5557):** Działa analogicznie do strumienia wideo (wzorzec PUB/SUB), przysyłając z robota do laptopa struktury danych JSON. Pakiety te zawierają zbiór krytycznych informacji diagnostycznych zebranych w czasie rzeczywistym, takich jak: napięcie głównego zasilania akumulatorowego, temperatury i prądy poszczególnych serwomechanizmów, aktualne kąty w stawach oraz logi systemowe.

Warto zaznaczyć, że biblioteka ZeroMQ została wykorzystana nie tylko do komunikacji zewnętrznej, ale również wewnątrz samego środowiska robota. W celu separacji logicznej modułów, zarządzanie ramieniem oraz platformą jezdną rozdzielono na niezależne procesy (*Arm Service* oraz



*Chassis Service*). Komunikują się one ze sobą lokalnie poprzez dedykowane, wewnętrzne porty (5558 i 5559), tworząc bezpieczną i odporną na błędy architekturę mikrousług (ang. *microservices*).

### 5.3 Architektura wieloprocasowa i nadzorca systemu (Supervisor)

Implementacja złożonego systemu zrobotyzowanego w środowisku języka Python wiąże się z koniecznością ominięcia ograniczeń wynikających z globalnej blokady interpretera (ang. *Global Interpreter Lock* – GIL). Wykonywanie intensywnych operacji wejścia/wyjścia (takich jak akwizycja wideo), ciągle nasłuchiwanie portów sieciowych oraz przeliczanie kinematyki w pojedynczym procesie doprowadziłoby do znacznych opóźnień i utraty płynności sterowania. Z tego względu na pokładzie jednostki *Controller Node* wdrożono architekturę wieloprocasową z centralnym węzłem zarządzającym.

Główny skrypt startowy robota (`controller_main.py`) nie realizuje bezpośrednio żadnych zadań percepcyjnych ani wykonawczych. Pełni on wyłącznie funkcję systemowego nadzorcy (ang. *Process Supervisor*). Do jego zadań należy asynchroniczne uruchomienie i monitorowanie trzech kluczowych, niezależnych podprocesów:

- **Arm Service:** moduł odpowiedzialny za komunikację ZMQ ze stacją bazową, obliczanie kinematyki oraz wysterowanie sterownika ESP32 ramienia.
- **Camera Service:** moduł dedykowany wyłącznie do sprzętowej akwizycji klatek obrazu z interfejsu USB, ich kompresji oraz strumieniowania w sieć.
- **Chassis Service:** moduł zarządzający komunikacją szeregową z mikrokontrolerem platformy jezdnej UGV02.

Takie podejście architektoniczne gwarantuje ścisłą izolację błędów (ang. *fault isolation*). Awaria lub chwilowe zawieszenie się procesu obsługującego kamerę nie wpływa w żaden sposób na proces obliczający kinematykę ramienia, co pozwala na zachowanie ciągłości sterowania fizycznego.

Co więcej, nadzorca systemu implementuje deterministyczny mechanizm reagowania na awarie krytyczne. W głównej pętli programu nieustannie weryfikowany jest status cyklu życia wszystkich uruchomionych podprocesów (metoda `poll()`). W przypadku wykrycia niespodziewanego zakończenia pracy któregoś z serwisów (np. z powodu rozłączenia fizycznego przewodu), nadzorca natychmiastowo inicjuje procedurę awaryjnego zatrzymania systemu (ang. *emergency shutdown*). Polega ona na bezpiecznym wysłaniu sygnałów terminacji (SIGTERM) do pozostałych działających modułów oraz poprawnym zamknięciu współdzielonego pliku logów (`robot.log`), który gromadzi diagnostyczne wyjścia standardowe (`stdout/stderr`) ze wszystkich podprocesów.

Zastosowanie architektury wieloprocasowej nadzorowanej przez nadrzędny skrypt kontrolny sprawia, że oprogramowanie robota jest nie tylko wydajne i wolne od blokad (ang. *non-blocking*), ale przede wszystkim wysoce odporne na błędy, co stanowi fundamentalny wymóg w projektowaniu nowoczesnej robotyki mobilnej.

## **6 Oprogramowanie pokładowe robota**

- 6.1 Sterownik serwomechanizmów i odczyt telemetry z magistrali**
- 6.2 Implementacja kinematyki analitycznej manipulatora**
- 6.3 Ograniczenia przestrzeni roboczej i sprzętowe limity bezpieczeństwa**
- 6.4 Akwizycja i kompresja strumienia wideo z kamery pokładowej**
- 6.5 Sterowanie platformą jezdnią**

## **7 Implementacja modułu sztucznej inteligencji**

### **7.1 Przygotowanie i konfiguracja modelu detekcji YOLO**

### **7.2 Przetwarzanie obrazu w czasie rzeczywistym i augmentacja danych**

## **8 Stworzenie stacji operatorskiej**

### **8.1 Projekt graficznego interfejsu użytkownika (GUI)**

### **8.2 Integracja bezprzewodowego kontrolera i mapowanie przycisków**

### **8.3 Wizualizacja telemetry i synchronizacja danych**

## **9 Testy funkcjonalne i weryfikacja systemu**

### **9.1 Weryfikacja działania układu jezdnego**

### **9.2 Testy kinematyki ramienia i precyzji manipulacji**

### **9.3 Ewaluacja wydajności sieciowej i opóźnień (latency)**

### **9.4 Skuteczność algorytmów rozpoznawania obrazu**

## **10    Wnioski i kierunki dalszego rozwoju**

# Literatura

- [1] P. A. M. Dirac, *The Principles of Quantum Mechanics* (International series of monographs on physics). Clarendon Press, 1981, ISBN: 9780198520115.
- [2] A. Einstein, „Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies],” *Annalen der Physik*, t. 322, nr 10, s. 891–921, 1905. doi: <http://dx.doi.org/10.1002/andp.19053221004>
- [3] B. Siciliano i O. Khatib, *Springer Handbook of Robotics*, 2nd. Springer, 2016, ISBN: 9783319325521.
- [4] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th. Pearson, 2018.
- [5] I. Goodfellow, Y. Bengio i A. Courville, *Deep Learning*. MIT Press, 2016.
- [6] J. Redmon, S. Divvala, R. Girshick i A. Farhadi, „You only look once: Unified, real-time object detection,” w: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, s. 779–788.
- [7] R. Siegwart, I. R. Nourbakhsh i D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. Cambridge, Massachusetts: MIT press, 2011.

## Summary